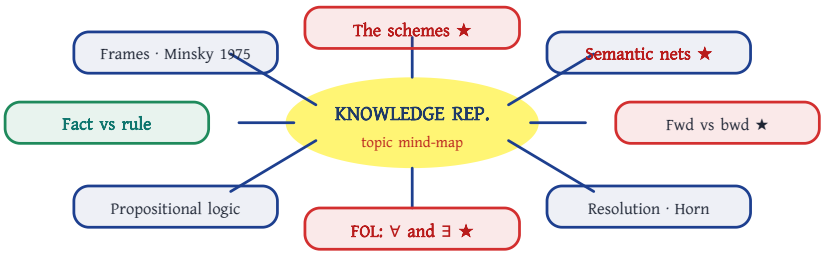
Knowledge Representation & Logic

Schemes · propositional & first-order logic · unification · resolution · chaining: exam-ready in 12 sheets



↑ the whole topic on one page: red = high-yield, green = the one idea everything hangs on

Syllabus coverage

The Artificial Intelligence syllabus lists "Knowledge representation" and "Propositional and first order logic" as two separate lines. This notebook covers both, plus the inference machinery every exam paper on the topic assumes. Search and probabilistic reasoning are the neighbouring lessons.

High-yield

① KR = formal encoding, so a machine can reason
② Six schemes: logic, net, frame, rules, script, ontology
③ Quillian 1968 · Minsky 1975 · Schank & Abelson 1977 · Robinson 1965
④ P→Q false only when P true, Q false
⑤ Modus ponens: P, P→Q ⊢ Q
⑥ Resolution: (P∨Q), (¬P∨R) ⇒ (Q∨R)
⑦ Horn = one positive literal at most
⑧ Forward = facts→goal, backward = goal→facts

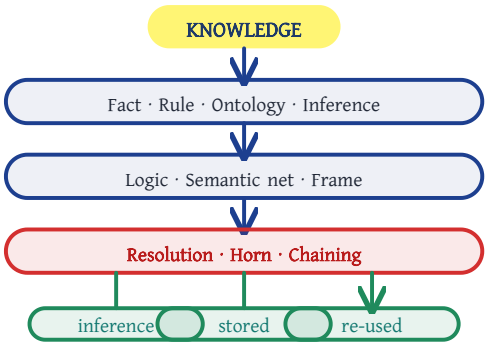
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ What knowledge representation is … p. 2
✓ The representation schemes … p. 3
✓ Semantic networks & inheritance … p. 4
✓ Frames and scripts … p. 5
✓ Propositional logic: syntax … p. 6
✓ Inference, validity, tautology … p. 7
✓ First-order logic & quantifiers … p. 8
✓ Unification, resolution, Horn … p. 9
✓ Forward vs backward chaining … p. 10
✓ Night-before p. 11 · Practice p. 12

1 What is Knowledge Representation?

Definition: **Knowledge Representation (KR)** is the process of **formally encoding** information about the world in a form an AI system can store and use to **reason, infer and decide**. The knowledge is explicit and machine-processable instead of being hidden inside program code.



representation → logic → resolution → inference

The four building blocks: **Fact** — a basic assertion, e.g. *Human(Socrates)* · **Rule** — a conditional linking facts, e.g. *Human(x)* → *Mortal(x)* · **Ontology** — the formal set of concepts and their relations in a domain · **Inference** — **deriving new knowledge from existing facts and rules**

KR vs	Answer
data	KR gives data **structure and meaning**
AI	KR is a **part** of AI; AI also searches, learns
a database	a database **stores**; KR permits **inference**

Property a good scheme needs	The question it answers
Representational adequacy	can it describe everything the domain needs?
Inferential adequacy	can the required conclusions be derived?
Inferential efficiency	how fast is a conclusion produced?
Acquisitional efficiency	how cheap is it to add a new fact?
Ease of acquisition	can a human read, check and correct it?

Memory trick: "**Representation on the left, reasoning on the right.**" KR decides *how knowledge is stored*; the inference engine decides *how it is used.*

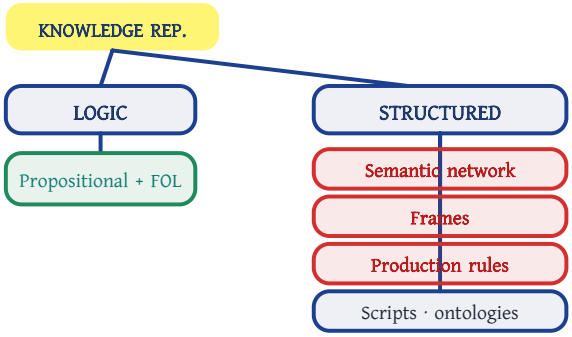
Exam trap: KR is **not** the same as knowledge, nor a database. "Knowledge" here is **declarative** — a statement of what is true — while the procedure that uses it belongs to the inference engine.

Worked example ✍️ the smallest possible expert system:
Knowledge base: fact *Human(Socrates)*, rule *Human(x)* → *Mortal(x)*. Unify **x with Socrates**, apply the rule, get *Mortal(Socrates)*, then **store the conclusion back** in the base — that loop is inference.

Why it matters in practice: Two systems can hold identical facts and behave completely differently because their representation differs. A rule base lets you explain every answer; a neural network cannot. When an exam asks for two desirable properties, the table above is the answer.

2 The Knowledge Representation Schemes

Definition: A **KR scheme** is the notation and structure used to encode knowledge. Every scheme is a compromise: it says a great deal about some kinds of fact and very little about others. The classic comparison has **six schemes** — logical, semantic network, frame, production rule, script and ontology.



two families: logic gives proof, the structured schemes give structure

Where they appear in real systems: Semantic networks and ontologies are the same idea at two scales — an early AI representation, and the machinery of the Semantic Web (RDF, OWL) and of modern knowledge graphs. Frames are the ancestor of every class-based object model; production rules of every expert system.

Scheme	Core idea	Strength	Weakness
Logical (PL, FOL)	facts and rules as formulas	exact and provable	verbose, no partial knowledge
Semantic network	nodes = concepts, arcs = relations	facts inherit automatically	no formal semantics, cannot prove
Frame	a stereotyped object: slots, facets	handles defaults and exceptions	rigid and verbose, no computation
Production rule	IF–THEN rules on a working memory	modular, easy to extend	slow, needs a conflict strategy
Script	a stereotyped sequence of events	reads situations well	cannot compute, no surprises
Ontology	axioms over classes, vocabulary	reuse and consistency	expensive to build and maintain

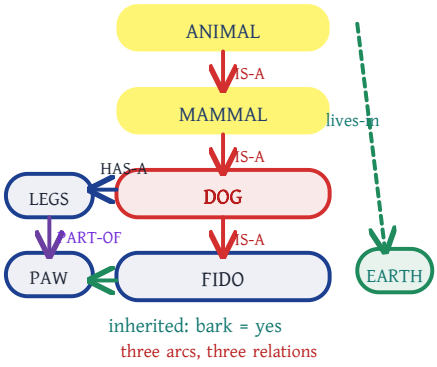
Memory trick: Match the **question** to the scheme: *is it true?* → logic · *how related?* → net · *what does it have?* → frame · *what if...?* → rule · *in what order?* → script · *what vocabulary?* → ontology.

Exam trap: An ontology is **not** a sixth cousin of the others — it is **the vocabulary plus the constraints** they are written in. A script is **not** a plan: a script is what usually happens, a plan is what this agent intends.

Worked example ✍ pick a scheme: "birds fly" → **frame** (default + exception penguin). "if temperature > 100 °C then alarm" → **production rule**. "enter → order → eat → pay → leave" → **script**. "if it rains the ground is wet" → **propositional logic**. "dog is-a animal" → **semantic network**.

3 Semantic Networks & Inheritance

Definition: A **semantic network** represents knowledge as a **directed graph** whose **nodes are objects, concepts, events or properties** and whose **labelled arcs are the relations between them**. Introduced by **M. R. Quillian, 1968** as a model of associative memory; the direct ancestor of the knowledge graph.



facts are not stored on Fido; they are inherited down the IS-A chain

Arc family	Typical labels
Structural	IS-A, HAS-A, PART-OF, AKO
Semantic	synonym, antonym, hyponym
Procedural	arcs attached to an action

Inheritance: An object takes the properties of its class without restating them. Two kinds: **default inheritance** (assume the general case unless told otherwise) and **specific inheritance** (an override always wins). Inheritance is **transitive** — it travels down the whole chain.

HAS-A means "has a part", IS-A means "is a kind of" — swap the two and the label flips!

Property	Semantic network
Facts inherit for free	**yes** — add a property to a class and every subclass gains it
Formal semantics	**none** — the arcs have no logical meaning
Consistency	not guaranteed — two arcs can contradict
Inference	traversal only: **inheritance** and graph search

Memory trick: **"IS-A is a class arrow, HAS-A is an ownership arrow."** You may say "a dog IS-A mammal"; you may never say "a mammal IS-A dog". Reversing an IS-A arc is the commonest mistake in an exam diagram.

Exam trap: A semantic network is **not** a truth table and **not** a production system — it answers "what is related to what", not "what follows from what". And it cannot **prove** anything.

Worked example ✍️ inheritance with an exception: Store *Animal → breathes = true*, *Mammal IS-A Animal*, *Dog IS-A Mammal*, *Fido IS-A Dog*, plus *Fido → legs = 4*. Ask for *Fido → breathes*: the engine walks the chain **Fido → Dog → Mammal → Animal** and answers **true** without Fido ever storing it. Now add *Fido → legs = 3* — the **specific** value 3 must win, which is exactly the conflict-resolution problem frames solve explicitly.

4 Frames and Scripts

Frame — Minsky, 1975: A **frame** describes a **stereotyped object, class or situation** as a named set of **slots**. Each slot holds a value, a **facet** (default, range, if-needed procedure) or a link to another frame. A frame for a class is a **schema**; filling one in gives an **instance**.

Script — Schank & Abelson, 1977: A **script** is a stereotyped **ordered sequence of events** that normally takes place in a situation. It stores what is *typical*, so a story becomes predictable and cheap to understand.

FRAME: CAR		
slot	value	facet
colour	red	def. white
wheels	4	range 2–8
engine	petrol	if-needed
owner	—	if-added

name · slots · values · facets — the facets are the whole point of a frame

The five parts of a script: ① **initial conditions** ② a sequence of **states and events** ③ **transitions**, each with a condition so steps can be skipped ④ **resulting conditions** ⑤ **props** — the roles played by objects: *customer*, *waiter*, *food*. A frame instead has a **name, slots, values and facets** — no time line at all.

Point of comparison	Frame	Script
What it describes	an **object or class** and its attributes	a **situation** and its events in order
Internal structure	name + slots + values + facets	initial + states + transitions + props
Organised by	a **class / instance** hierarchy	a **time line** of states
Handles exceptions	yes — **default overridden**	partly — a step may be skipped

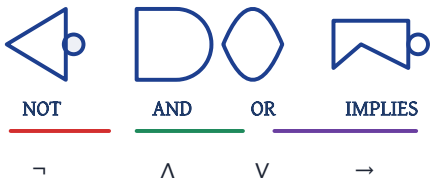
Memory trick: "**Frame = furniture, script = theatre.**" A frame is a thing you point at; a script is something that happens to it in a fixed order. Facets live only in frames; props live only in scripts.

Exam trap: Frames are **not** a record format with typed columns — a slot may hold a value, a **default**, a range or a **procedure**. And a script is **not** a production rule: a rule is a guarantee, a script is only an expectation.

Worked example ✍ a frame doing real work: Open the frame *CAR* and ask for the colour of "my car". No slot has been filled, so the engine takes the **default facet** and answers **white**. Ask for the engine of a car that has been scrapped: the *if-needed* procedure runs instead of returning a value. That is why frames are said to carry **procedural attachments**.

5 Propositional Logic: Syntax & Connectives

Definition: Propositional (or sentential) logic treats every whole statement as a single **proposition** that is either true (T) or false (F), joined by **logical connectives**. An **atomic** proposition is a single letter P; a **compound** proposition is built from atoms and connectives.



the four connectives drawn as gates

Connective	Symbol	True exactly when
negation, NOT	¬P	P is false
conjunction, AND	P ∧ Q	both P and Q are true
disjunction, OR, inclusive	P ∨ Q	at least one of P, Q is true
implication, IF–THEN	P → Q	**anything except P true and Q false**
biconditional, IFF	P ↔ Q	P and Q have the same truth value

P	Q	P ∧ Q	P ∨ Q	¬P	P → Q	P ↔ Q
T	T	T	T	F	T	T
T	F	F	T	F	F	F
F	T	F	T	T	T	F
F	F	F	F	T	T	T

Precedence, tightest first: ¬ → ∧ → ∨ → → → ↔. Brackets beat everything: ¬(P ∧ Q) is **not** the same as (¬P) ∧ Q.

Limitation: Propositional logic **cannot** express variables, quantifiers or relations between objects. Nothing is shared between "Socrates is human" and "Plato is human" — the base grows as a flat list and inference cannot generalise.

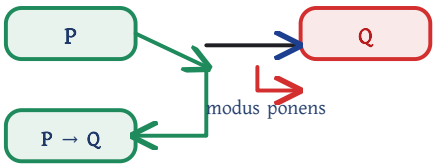
Worked example ✍ build and read a formula: "If it rains and I carry an umbrella, the ground stays dry." Let R = rain, U = umbrella, D = ground dry. The formula is **(R ∧ U) → D**. If R = T, U = T, D = F the implication is **false** — the one and only combination that breaks it. If R = F the formula is true whatever else happens: an implication is **not** a biconditional.

Memory trick: "**→ only looks backwards once.**" P → Q is false in exactly one row, so read it as "everything is fine *unless* P happens and Q does not." Write P → Q as **¬P ∨ Q** and it becomes obvious.

Exam trap: (i) OR is **inclusive**: P ∨ P = P, not P. (ii) P → Q is **not** P ← Q. (iii) "P is false and Q is true" makes P → Q **true**, not false. (iv) A proposition is an entire sentence — you may not split "Socrates is a man" into two.

6 Validity, Satisfiability & Inference Rules

Definitions — all four are about the truth table: A **tautology (valid)** formula is true for **every** assignment · a **contradiction (unsatisfiable)** is false for every assignment · a **satisfiable** formula is true for **at least one** assignment. "P ⊨ Q" means the truth table of **P → Q** is a tautology. n propositions need **2^n rows**.



modus tollens: ¬Q, P → Q ⇒ ¬P

fact + rule → new fact: the only pattern every expert system runs

Rule of inference	From	Conclude
Modus ponens	P, P → Q	**Q**
Modus tollens	¬Q, P → Q	**¬P**
Hypothetical syllogism	P → Q, Q → R	**P → R**
Disjunctive syllogism	P ∨ Q, ¬P	**Q**
Addition	P	**P ∨ Q**
Simplification	P ∧ Q	**P**
Resolution	P ∨ Q, ¬P ∨ R	**Q ∨ R**

Worked example 1 ✍ classify four formulas:
(a) **P ∨ ¬P** — T in every row ⟹ **tautology**. (b) **P ∧ ¬P** — F in every row ⟹ **contradiction**. (c) **P → (Q ∨ ¬Q)** — consequent always true ⟹ tautology. (d) **(P ∨ Q) ∧ ¬P** — true when P = F, Q = T ⟹ satisfiable, not valid.

Worked example 2 ✍ modus tollens in the clinic:
From $P \to Q$ = "if the patient took the drug, the rash clears" and $\neg Q$ = "the rash has not cleared", modus tollens gives **¬P**: the patient did not take the drug.

Why resolution wins: Modus ponens needs a matching fact at every step; resolution needs only two clauses, and it alone works on disjunctions — hence automated theorem proving (Robinson, **1965**).

Memory trick: "**Tautology always, satisfiable sometimes, contradiction never.**" Tautology = true *always*; contradiction = true *never*.

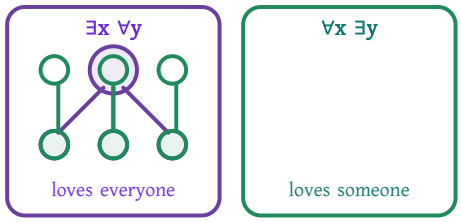
Exam trap: "Valid" and "true" are **not** the same: valid only if *every* assignment makes it true. P ⊨ Q does **not** give Q.

7 First-Order Logic, Predicates & Quantifiers

Definition: **First-order logic (FOL)** extends propositional logic with **objects, predicates over them, functions and quantifiers**. A formula is built from atomic formulas $P(t_1,\ldots,t_n)$ using connectives and quantifiers.

Component	What it is	Example
Constant	one named individual	Socrates
Variable	an arbitrary individual	x, y
Function	individual → individual	FatherOf(x)
Predicate	property or n-place relation	Human(x), Loves(a,b)
Quantifier	ranges the variable	∀x, ∃y

The two quantifiers: ∀ "for **all** x ..." · ∃ "there **exists** x such that ...". **Their order matters.** A **WFF** is any well-formed formula; a **sentence** is a WFF with no free variables.



the quantifier order changes the meaning

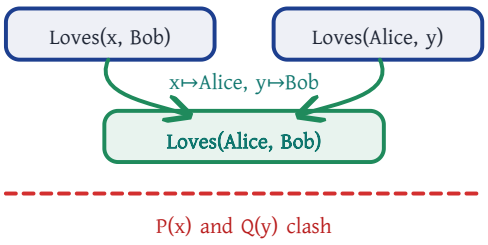
English sentence	First-order form	Feature	Propositional	First-order
All humans are mortal	∀x (Human(x) → Mortal(x))	Talks about	whole statements	objects, relations
Someone loves everyone	∃x ∀y Loves(x, y)	Quantifiers	none	**∀ and ∃**
Everyone loves someone	∀x ∃y Loves(x, y)	Cost	cheap, 2^n rows	**expensive**

Memory trick: "**∀ says every, ∃ says at least one.**" The → **inside** the ∀ is what keeps "all humans are mortal" true even when some x is not a human: Human(x) → Mortal(x) never asserts Mortality of a non-human.

Exam trap: ∀x P(x) is **not** P(a). And "all humans are mortal" must **not** be written ∀x (Human(x) ∧ Mortal(x)) — that would claim everybody is both human and mortal.

Worked example ✍ why the order matters: Committee rule: "every member has at least one car" = ∀x (Member(x) → ∃y (Car(y) ∧ Owns(x,y))). The two quantifiers can pick **different** cars for **different** members, so the base stays consistent — but as soon as the committee shrinks to **one member and no cars**, ∀x∃y fails while its negation is still available, which is exactly the case a resolution engine detects automatically.

8 Unification, Resolution & Horn Clauses



unify before you match: that is all inference in one picture

Unification: **Unification** is finding a substitution for the variables of two atoms so that they become identical. The result is the **most general unifier (MGU)**; when no substitution exists the two atoms **clash**.

Algorithm: Walk the atoms left to right. Equal constants → go on. A variable on either side → **bind it to the other term**. Different constants → clash. Never bind a variable to a term that already contains it (occurs check): X = f(X) has no solution.

Atom pair	Most general unifier
Loves(x, Bob) , Loves(Alice, y)	{x↦Alice, y↦Bob}
Human(x) , Human(Socrates)	{x↦Socrates}
P(x) , Q(y)	**clash — none**
P(x, y) , P(y, z)	{y↦x, z↦x}

Refutation by resolution: To prove goal φ, convert every statement to **clause form**, add the **negated** goal, and resolve clauses repeatedly. Two clauses that resolve to the **empty clause** prove the goal. Sound, and complete for FOL by Robinson's theorem.

Worked example ✍️ prove Mortal(Socrates) by resolution:

1. KB: Human(Socrates) · Human(x) → Mortal(x)
2. Clause form: **C_1 = Human(Socrates)**, **C_2 = ¬Human(x) ∨ Mortal(x)**; add the negated goal **C_3 = ¬Mortal(Socrates)**
3. Resolve C_1 with C_2 — *Human* and ¬*Human* disappear: **C_4 = Mortal(Socrates)**
4. Resolve C_4 with C_3 — *Mortal* and ¬*Mortal* disappear ⟹ **the empty clause**, so the goal is proved

Memory trick: "**Prove by not-proving.**" Refutation: add ¬goal and search for a contradiction. Resolution is the only rule that consumes a **disjunction**, and every formula can be turned into a disjunction of disjunctions.

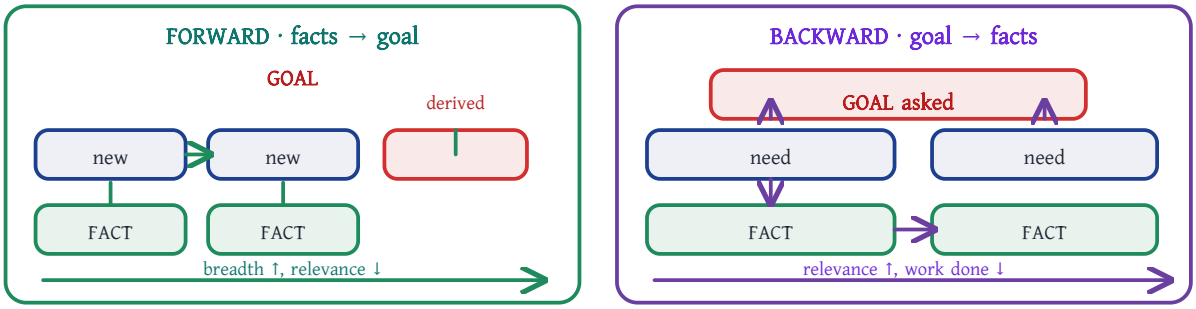
Exam trap: Resolving (P ∨ Q) with (¬P ∨ R) gives **Q ∨ R** — you drop the **complementary** pair P, ¬P and keep both remaining literals joined by ∨, never ∧. Horn clauses, next box, cannot express a disjunction in the head at all.

Horn clauses — why the exam loves them: A **Horn clause has at most one positive literal**, i.e. H ← B_1 ∧ B_2 ∧ … ∧ B_n. Three consequences: **forward chaining is complete** for Horn knowledge · **backward chaining is complete** as well · the logic is **monotonic** — adding facts never removes conclusions. Horn logic is exactly what Datalog and every production system run on.

9 Forward and Backward Chaining

Forward chaining: **Data-driven, bottom-up.** Start from the facts, fire every rule whose antecedents are satisfied, and add each conclusion to the working memory. Repeat until nothing fires.

Backward chaining: **Goal-driven, top-down.** Start from the goal, find the rule whose head matches it, and recurse on that rule's antecedents until only facts remain.



both use the same rules — only the direction of travel differs

Point of comparison	Forward chaining	Backward chaining
Starts from	the facts (data-driven)	the goal (goal-driven)
Answers	**everything** derivable	**only** what the goal needs
Cost	may explode: many firings, irrelevant facts	cheap when the goal is specific
Explainability	full — you can print the whole chain	partial — only the proof needed
Best when	monitoring, all facts known, many questions	diagnosis, one clear question, missing facts
Complete for	Horn knowledge	Horn knowledge

Memory trick: "**Forward looks up, backward looks down.**" Forward climbs from data to conclusions; backward descends from the question to the data. "Forward is thorough, backward is targeted" is the one-line answer to the written question.

Exam trap: Backward chaining does **not** invent facts — if the knowledge base does not support the goal it simply **fails**; it never guesses. When several rules fire at once the engine needs a **conflict-resolution strategy** (breadth-first, depth-first, most-specific, most-recent).

Worked example ✍️ choose the direction: **(a)** A bank must flag every transaction above 10 000 taka *as it happens* — all facts are known and there is no single question: **forward chaining**. **(b)** A doctor types "diagnosis?" and the system must justify one answer from thousands of symptoms: **backward chaining**. **(c)** The knowledge base contains a disjunction P ∨ Q, which is not Horn: neither chaining is complete, and **resolution** is the answer.

Night-Before-the-Exam Sheet

Fact sheet:

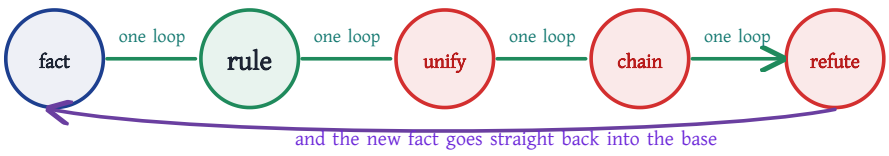
KR = formal encoding of information for reasoning	Six schemes: logic, net, frame, rules, script, ontology
Quillian 1968 · Minsky 1975 · Schank & Abelson 1977	Robinson 1965 (resolution)
Semantic net: nodes + labelled arcs, inheritance	Frame: slots, values, facets, defaults
Script: initial, states, transitions, result, props	Ontology: vocabulary + axioms
Connectives: ¬ ∧ ∨ → ↔	P → Q false only when P = T, Q = F
Precedence: ¬ > ∧ > ∨ > → > ↔	P → Q ≡ ¬P ∨ Q ≡ contrapositive ¬Q → ¬P
Modus ponens P, P→Q ⊢ Q	Modus tollens ¬Q, P→Q ⊢ ¬P
Resolution: (P∨Q), (¬P∨R) ⇒ (Q∨R)	Tautology = always true · contradiction = never
∀x P(x) · ∃x P(x) · order matters	Horn clause: at most one positive literal
Forward = facts → goal · Backward = goal → facts	Both complete for Horn; Horn is monotonic

Top 12 one-liners for MCQs

1. Semantic networks: **Quillian, 1968**
2. Frames: **Minsky, 1975**
3. Scripts: **Schank & Abelson, 1977**
4. Resolution: **Robinson, 1965**
5. IS-A only goes subclass → superclass
6. A semantic network has **no formal semantics**
7. A frame carries **defaults and procedures**
8. A script stores only what **usually** happens
9. Forward chaining: data-driven, **bottom-up**
10. Backward chaining: goal-driven, **top-down**
11. Resolution is **complete** for FOL by Robinson
12. Horn = **at most one positive literal**

Top traps: P → Q is true when P is false · OR is inclusive · resolution keeps ∨, never ∧ · modus tollens needs ¬Q, not P · ∃x ∀y ≠ ∀x ∃y · "all humans are mortal" uses → inside ∀, not ∧ · a script is not a plan · a frame is not a typed table row · backward chaining never guesses.

All tricks together: "Representation left, reasoning right" · "IS-A class arrow, HAS-A ownership arrow" · "Frame = furniture, script = theatre" · "→ only looks backwards once" · "Tautology always, satisfiable sometimes, contradiction never" · "∀ says every, ∃ says at least one" · "Prove by not-proving" · "Forward looks up, backward looks down".



fact + rule → unify → chain → refute, and the loop starts again

✓ Practice MCQs (exam pattern, 10 marks)

1. Knowledge representation is:

(a) a language (b) formal encoding for reasoning

(c) a search algorithm (d) a database engine

2. A semantic network uses:

(a) nodes and labelled arcs (b) truth-table rows

(c) IF–THEN rules only (d) slot tables only

3. Frames are associated with:

(a) Quillian (b) Robinson

(c) Schank (d) Minsky

4. $P \rightarrow Q$ is false only when:

(a) P false, Q true (b) both false

(c) P true, Q false (d) both true

5. From P and $P \rightarrow Q$, concluding Q uses:

(a) modus tollens (b) modus ponens

(c) hypothetical syll. (d) disjunctive syll.

6. $P \rightarrow Q$ is equivalent to:

(a) $Q \rightarrow P$ (b) $\neg P \rightarrow \neg Q$

(c) $\neg Q \rightarrow \neg P$ (d) $P \wedge \neg Q$

7. Resolving $(P \vee Q)$ with $(\neg P \vee R)$ gives:

(a) $(Q \vee R)$ (b) $(P \vee R)$

(c) $(\neg Q \vee R)$ (d) $(Q \wedge R)$

8. "All men are mortal" is:

(a) $\exists x\ (Man(x) \rightarrow Mortal(x))$ (b) $\forall x\ (Man(x) \rightarrow Mortal(x))$

(c) $\forall x\ (Man(x) \wedge Mortal(x))$ (d) $\exists x\ Man(x) \wedge \exists x\ Mortal(x)$

9. A Horn clause has at most:

(a) one negative literal (b) one positive literal

(c) one variable (d) one predicate

10. Forward chaining begins from:

(a) the goal (b) the rules

(c) the facts in memory (d) the refutation

Answer key → 1-b · 2-a · 3-d · 4-c · 5-b · 6-c · 7-a · 8-b · 9-b · 10-c

Written question: Define knowledge representation and name four schemes with one strength and one weakness each. Then for the KB *Human(Socrates)* and *Human(x) → Mortal(x)*: (a) write the first-order form, (b) convert it to clause form, (c) use resolution to prove Mortal(Socrates), (d) is forward or backward chaining better here, and why? 10

1. **Definition and schemes (3)** — KR is the formal encoding of information so that an AI system can reason over it. Schemes: logic (exact, no partial knowledge) · semantic network (inherits facts, no formal semantics) · frame (defaults, but rigid) · production rules (modular, but slow). 3
2. **First-order form (1)** — Human(Socrates) and $\forall x\ (Human(x) \rightarrow Mortal(x))$. 1
3. **Clause form (2)** — $\neg Human(x) \vee Mortal(x)$, quantifier dropped: C_1 = Human(Socrates), $C_2 = \neg Human(x) \vee Mortal(x)$. 2
4. **Resolution (3)** — add negated goal $C_3 = \neg Mortal(Socrates)$; C_1 with $C_2 \rightarrow Mortal(Socrates)$; that with $C_3 \rightarrow$ empty clause, goal proved. 3
5. **Direction (1)** — both are complete for this Horn base; backward chaining, because one question is asked. 1

✓ Can define KR, list five schemes

✓ Can translate a sentence into ∀ / ∃

✓ Can build a truth table for → and ↔

✓ Can unify two atoms, resolve two clauses

✓ Can state the chaining trade-off

✓ Knows what a Horn clause is